

Rapport — Java Avancé

Membres du groupe :

Kwassi Nyuieva ABOTSI - Ahmed CHTIOUI - Antoine FONTANEZ - Nathan PIERROT

Année 2025–2026

Table des matières

Introduction	5
Architecture générale du projet	6
Vue d'ensemble	6
Schéma de communication	6
Répartition du travail	7
Partie back-end	8
Le protocole MQTT	8
Présentation du protocole	8
Les topics utilisés	8
Format des messages	8
Les classes du module server	9
Main	9
Serveur	10
MessageManager	10
MessageParser	11
ErreurMessage	11
Le module factory (l'Usine)	12
Rôle de l'Usine	12
Architecture interne	12
La classe Usine	12
La classe Demande	13
La classe Consommateur	13
Algorithme de production	13
Gestion de la concurrence	14
Diagramme de séquence	15
Tests unitaires du back-end	15
Réponses aux questions du sujet	16
Comment gérer l'absence d'usine connectée ?	16
Gérer plusieurs commandes en parallèle pour un même client	16
Gérer plusieurs usines	17
Ce qui reste à améliorer	17
Partie front-end - Les vues JavaFX	18
Les maquettes	18
Présentation des écrans	20
La page d'accueil	20
La page de catalogue	20
La page de fabrication	21
La page de vérification	22
Articulation des pages et navigation	23
La classe Layout	23
La navigation	24

La navigation depuis les vues elles-mêmes	24
Lancement de l'application JavaFX	24
La classe Lanceur	24
La classe Main	25
Partie front-end - Architecture MVC et communication réseau	26
Architecture MVC.....	26
Le patron MVC	26
Organisation des packages	26
Les classes de Modèle	27
GlassesModel	27
Order	27
SerialPair	28
Les Contrôleurs	28
AppController	28
OrderController	28
SerialController.....	30
La service ClientMqtt	30
Présentation du protocole MQTT	30
Synchronisation par BlockingQueue	30
Gestion des timeouts.....	31
Tests unitaires du front-end	31
Stratégie de test	31
Tests de Order	31
Tests de OrderController	31
Tests de SerialController	32
Qualité du code.....	32
Javadoc.....	32
Principes de conception appliqués	32
Difficultés rencontrées	32
Synchronisation MQTT et thread appelant.....	32
Binding réactif sans couplage	33
Mock de construction dans les tests	33
Intégration continue.....	34
Objectif.....	34
Fonctionnement de la GitHub Action.....	34
Intégration avec Maven	35
Action de compilation continue.....	35
Bénéfices pour le projet	35
Bilan du projet.....	36
Ce qui a été réalisé.....	36
Les difficultés rencontrées	36
Ce qui pourrait être amélioré.....	37

Répartition du travail	37
Conclusion.....	37

Introduction

Ce rapport présente le travail que nous avons réalisé dans le cadre du projet Java Avancé : la création d'une application distribuée appelée « La Fabrique de Lunettes ». Cette application permet à un client de commander des lunettes connectées, de suivre leur fabrication et de recevoir les numéros de série correspondants une fois la production terminée.

Le projet a été réalisé en équipe de quatre personnes. Chaque membre a pris en charge une partie précise du système :

- Antoine FONTANEZ : la partie back-end, c'est-à-dire le serveur MQTT et le module de fabrication (l'usine).
- Kwassi Nyuieva ABOTSI : l'interface graphique JavaFX (les vues).
- Nathan PIERROT : l'architecture MVC, les contrôleurs et la communication MQTT côté client.
- Ahmed CHITIOUI : la mise en place de l'intégration continue avec GitHub Actions.

L'application repose sur plusieurs technologies :

- JavaFX pour l'interface graphique.
- MQTT (protocole publish/subscribe) pour la communication entre le client et le serveur.
- Maven pour la gestion des dépendances et la compilation du projet.
- Le patron de conception MVC pour organiser le code de façon claire.

Ce rapport est organisé de la façon suivante. Nous commençons par présenter l'architecture générale du projet, puis nous détaillons chaque grande partie : le back-end (serveur + usine), le front-end (interface graphique + architecture MVC + communication réseau), et enfin les tests unitaires et la mise en place de l'intégration continue.

Architecture générale du projet

Vue d'ensemble

Nous avons découpé le projet en plusieurs modules Maven indépendants. Cela nous a permis de travailler sur nos parties sans dépendre des autres, et de faciliter la compilation et les tests de chaque module séparément.

Les modules du projet sont les suivants :

- **factory** : contient les classe Usine, Consommateur et Demande qui pilotent le Fabricateur fourni dans le sujet. Ce module est compilé en fichier .jar et publié sur GitHub Packages à chaque publication d'une release.
- **server** : contient le serveur MQTT et toute la logique de traitement des commandes. Il dépend du module factory.
- **frontend** : contient l'interface graphique JavaFX, les modèles de données, les contrôleurs et le client MQTT qui permet de communiquer avec le serveur.

Les différentes parties du système communiquent entre elles grâce au protocole MQTT. Avec un broker Mosquitto qui tourne sur le port 1883 et qui sert de point d'entrée pour tous les échanges de messages.

Schéma de communication

Le schéma ci-dessous résume les interactions entre les différents composants :



Ce modèle publish/subscribe (publication/souscription) permet un couplage faible entre les composants : le client n'a pas besoin de connaître l'adresse du serveur directement, il passe par le broker. Si le serveur est temporairement arrêté, le client peut continuer à fonctionner sans planter.

Répartition du travail

Voici un tableau récapitulatif de la répartition des tâches :

Membre	Partie réalisée
Antoine FONTANEZ	Back-end : serveur MQTT, module factory, gestion de la concurrence, tests unitaires du back
Kwassi Nyuieva ABOTSI	Front-end : toutes les vues JavaFX (accueil, catalogue, fabrication, vérification), la préparation des contrôleurs associés
Nathan PIERROT	Front-end : architecture MVC, contrôleurs, client MQTT, tests unitaires des contrôleurs
Ahmed CHITIOUI	Intégration continue : GitHub Actions pour la génération du .jar de l'usine

Partie back-end

Cette partie décrit l'architecture et l'implémentation du back-end du projet. Le back-end est composé de deux modules Maven indépendants : le module server, qui reçoit et traite les commandes des clients, et le module factory, qui gère la production des lunettes via le Fabricateur.

Le protocole MQTT

Présentation du protocole

MQTT est un protocole de communication léger basé sur le modèle publication/souscription (publish/subscribe). Un composant appelé broker (ici Mosquitto) reçoit tous les messages et les redistribue aux clients abonnés. Dans notre projet, le broker tourne sur le port TCP 1883 sans authentification nécessaire.

Ce système de communication présente un avantage important : le couplage est faible entre les composants. Le client et le serveur ne se connaissent pas directement — ils échangent uniquement via des sujets (topics) définis au départ.

Les topics utilisés

Un topic est une chaîne de caractères qui identifie un canal de communication. Le serveur s'abonne aux topics suivants pour recevoir les messages des clients :

Topic	Direction	Description
orders/{id}	Client → Serveur	Le client envoie une commande de lunettes
orders/{id}/validated	Serveur → Client	La commande est bien formée et acceptée
orders/{id}/cancelled	Serveur → Client	La commande est refusée (erreur de format)
orders/{id}/delivery	Serveur → Client	Les lunettes sont fabriquées, voici les numéros
orders/{id}/error	Serveur → Client	Une erreur s'est produite pendant la fabrication
serials/{serial}/check	Client → Serveur	Le client veut vérifier un numéro de série
serials/{serial}	Serveur → Client	Résultat de la vérification (type ou 'invalid')

Format des messages

Le sujet nous interdisait de sérialiser nos données avec des formats standards comme JSON ou XML pour communiquer à travers le serveur. Nous avons donc créé un format simple, qui nous

a permis de facilement sérialiser les données avant l'envoi et de les désérialiser lors de leur réception.

Format d'une commande (client → serveur) :

TYPE:QUANTITE;TYPE:QUANTITE

Exemple : une commande de 3 lunettes CLAUDE et 2 lunettes BANANA s'écrit :

CLAUDE:3;BANANA:2

Format d'une livraison (serveur → client) :

TYPE:SERIAL;TYPE:SERIAL

Exemple : deux lunettes fabriquées, une CLAUDE et une BANANA :

CLAUDE:CL-AQR65E-C8E11277;BANANA:BA-XY123Z-ABCD1234

Les classes du module server

Main

La classe Main est le point d'entrée du back-end. Elle charge le fichier de configuration config.properties, crée une instance de l'Usine et du Serveur, puis démarre le serveur MQTT. La configuration est chargée avec getResourceAsStream, ce qui garantit que le fichier est bien trouvé aussi bien pendant le développement que dans le .jar final.

Nous avons créé le fichier config.properties, que nous avons ajouté dans le dossier ressource, qui contient les paramètres suivants :

```
mqtt.url=tcp://localhost:1883
mqtt.clientId=backend
capacity=5
```

Externaliser la capacité du Fabricateur dans ce fichier nous permet de modifier la valeur facilement sans avoir à devoir rechercher sa définition dans le code et sans avoir à recompiler le projet.

Serveur

La classe Serveur gère la connexion au broker MQTT et les abonnements aux topics. Au démarrage, elle s'abonne à deux topics :

- orders/+ : reçoit toutes les commandes. Le symbole + correspond à n'importe quel identifiant de commande.
- serials/+/check : reçoit les demandes de vérification de numéros de série.

Le Serveur ne traite pas lui-même les messages, il délègue entièrement cette responsabilité à la classe MessageManager, qu'il enregistre comme callback MQTT.

MessageManager

MessageManager implémente l'interface MqttCallback et est donc notifié dès qu'un message arrive sur un topic auquel le serveur est abonné. À la réception d'un message, il détermine le type de topic (orders ou serials) et appelle la méthode correspondante.

Traitement d'une commande :

1. Le message est analysé par MessageParser. S'il est invalide, une réponse d'erreur est publiée sur orders/{id}/cancelled.
2. Si la commande est valide, une confirmation est publiée sur orders/{id}/validated.
3. La production est lancée via l'Usine.
4. Les lunettes fabriquées sont publiées sur orders/{id}/delivery.
5. En cas d'erreur pendant la production, un message d'erreur est publié sur orders/{id}/error.

Vérification d'un numéro de série :

La méthode statique Fabricateur.validateSerial() est utilisée. Elle vérifie l'authenticité du numéro de série donné par un calcul de checksum CRC32 et retourne ensuite le type de lunette correspondant, ou la chaîne « invalid » si le numéro n'est pas reconnu.

MessageParser

La classe MessageParser s'occupe de la lecture et de l'écriture des messages. C'est elle qui gère toute la logique de validation des commandes, avec ces méthodes :

- `parseType` : convertit une chaîne de caractères en type de lunette (TypeLunette). Lève une exception si le type n'est pas reconnu.
- `qteLegal` : vérifie qu'une quantité est comprise entre 0 inclus et 10 exclu.
- `parsePayload` : analyse un message complet, valide chaque entrée et vérifie que la quantité totale commandée est supérieure à zéro.
- `serializeLunettes` : transforme une liste de lunettes produites en chaîne de caractères au format TYPE:SERIAL. Type correspond au modèle de lunette produit et Serial le numéro de série associé à cette paire.

ErreurMessage

ErreurMessage est une énumération qui regroupe tous les messages d'erreur liés à la validation. Nous les avons regroupés tous en un seul et même endroit pour pouvoir les modifier plus facilement en cas de besoin et pour les comparer par référence dans les tests unitaires. La liste des messages actuellement disponibles sont décrit dans ce tableau récapitulatif :

Constante	Message d'erreur
FORMAT_INVALIDE	Format de commande invalide
TYPE_INCONNU	Type de lunette inconnu
QUANTITE_INVALIDE	Quantité invalide, elle doit être comprise entre 0 (inclus) et 10 (exclu)
QUANTITE_TOTALE_INVALIDE	Quantité totale invalide, elle doit être strictement supérieure à 0

Le module factory (l'Usine)

Rôle de l'Usine

L'Usine est le composant qui pilote le Fabricateur fourni dans le sujet. Elle reçoit en entrée une Map associant chaque type de lunette à la quantité à produire, et retourne en sortie la liste complète des lunettes fabriquées avec leurs numéros de série.

Notre classe Usine est thread-safe : sa méthode produire() peut être appelée simultanément par plusieurs threads, ce qui correspond au cas réel où plusieurs commandes arrivent en parallèle depuis des clients différents.

Architecture interne

Notre package Usine contient trois classes qui utilisent le pattern Producteur/Consommateur :

- La classe Usine.java qui joue le rôle de producteur. Elle reçoit les demandes de fabrication, les décompose en objets « Demande » et les dépose dans une BlockingQueue partagée.
- La classe Consommateur.java qui joue le rôle de consommateur. Elle s'exécute dans un thread dédié (démon), vide la file en continu et utilise le Fabricateur par lots.
- Et enfin la classe Demande.java qui encapsule une demande de fabrication. Elle contient le type de lunette à produire et un CompletableFuture qui est complété une fois que la lunette est fabriquée. Cela permet de communiquer entre le consommateur et le thread appelant qui attend le résultat.

La classe Usine

L'Usine est le point d'entrée du module. Son constructeur instancie le Fabricateur avec la capacité fournie, crée un ExecutorService avec cette même capacité, initialise la BlockingQueue partagée, puis démarre le Consommateur dans un thread. Une fois construite, l'Usine est prête à recevoir des appels concurrents à « produire() ».

Sa méthode publique « produire() » est la seule disponible depuis le reste de l'application. Elle prend en charge la décomposition de la commande en demandes unitaires, leur dépôt dans la queue et l'attente des résultats via les futurs - les détails de cette séquence sont décrits dans la section "Algorithme de production" ci-dessous.

La classe Demande

Chaque Demande contient deux champs :

- type : le TypeLunette à produire.
- futur : un `CompletableFuture<Fabricateur.Lunette>` qui sera renseigné une fois la fabrication terminée.

Ce mécanisme est central : la méthode `produire()` peut déposer les demandes dans la queue et se mettre en attente sur les futurs, tandis que le Consommateur travaille de son côté de façon totalement indépendante.

La classe Consommateur

Le Consommateur s'exécute en boucle dans son propre thread. Voici le détail du fonctionnement de chaque itération :

1. Attendre au moins une Demande avec `queue.take()`. Cela bloque le thread sans consommer de CPU tant que la queue est vide.
2. Compléter le lot courant en récupérant les demandes supplémentaires déjà disponibles dans la queue, jusqu'à atteindre la capacité max du Fabricateur.
3. Configurer le Fabricateur avec les types de lunettes du lot.
4. Lancer la fabrication de chaque lunette du lot en parallèle via l'`ExecutorService`, en complétant le `CompletableFuture` associé à chaque Demande.
5. Attendre que toutes les tâches du lot soient terminées avant de passer au lot suivant.

Le `CompletableFuture` est complété directement dans les tâches données à l'`ExecutorService`. S'il y a une exception pendant la fabrication, il est complété avec une exception, cela permet au thread appelant de propager l'erreur proprement.

Algorithme de production

La méthode `produire()` suit les étapes suivantes :

Étape 1 : Aplatir la map

La map est d'abord convertie en une liste de types à produire grâce à la méthode privée `listerLunettes()`. Par exemple, si la map contient `{CLAUDE=3, BANANA=2}`, on obtient la liste `[CLAUDE, CLAUDE, CLAUDE, BANANA, BANANA]`. Cela simplifie la suite du traitement, chaque élément de la liste correspond à exactement une Demande.

Étape 2 : Créer et déposer les demandes

Pour chaque type de la liste, une Demande est instanciée. Toutes les demandes sont ensuite ajoutées en une seule opération à la `BlockingQueue` partagée avec `queue.addAll()`. Le Consommateur, qui tourne en permanence dans son thread, prendra en charge ces demandes dès qu'elles seront disponibles.

Étape 3 : Attendre les résultats

La méthode `produire()` appelle ensuite `demande.futur.get()` pour chaque Demande, en bloquant jusqu'à ce que le Consommateur ait fabriqué la lunette correspondante et complété le futur. Les lunettes sont récupérées dans une liste qui est retournée à l'appelant.

Gestion de la concurrence

Le pattern producteur/consommateur est le point central du fonctionnement de la concurrence dans notre package factory. Nous allons détailler le rôle de chaque classe concernant l'exécution parallèle de la fabrication dans cette partie du rapport.

Du côté de l'Usine, nous n'avons implémenté aucun mécanisme de synchronisation, la `BlockingQueue` est de base thread-safe. Plusieurs threads peuvent appeler `produire()` en même temps et déposer leurs demandes dans la queue sans qu'il y ait de problème.

Pour le Consommateur, un seul thread gère le Fabricateur à la fois. Le regroupement par lots a un rôle important, si plusieurs commandes arrivent quasiment au même moment, leurs demandes respectives peuvent être rassemblées dans un même lot et fabriquées en parallèle en une seule passe du Fabricateur. Cela nous permet de d'utiliser la capacité du Fabricateur au maximum et de réduire à la fois le nombre d'appels et le temps de fabrication.

Nous avons lancé le thread du Consommateur en tant que thread démon (`thread.setDaemon(true)`) pour qu'il ne bloque pas l'arrêt de la JVM si tous les threads utilisateurs ont terminé leur exécution.

Diagramme de séquence

Voici un diagramme de séquence pour avoir un aspect plus visuel et plus synthétique du fonctionnement de notre Usine :

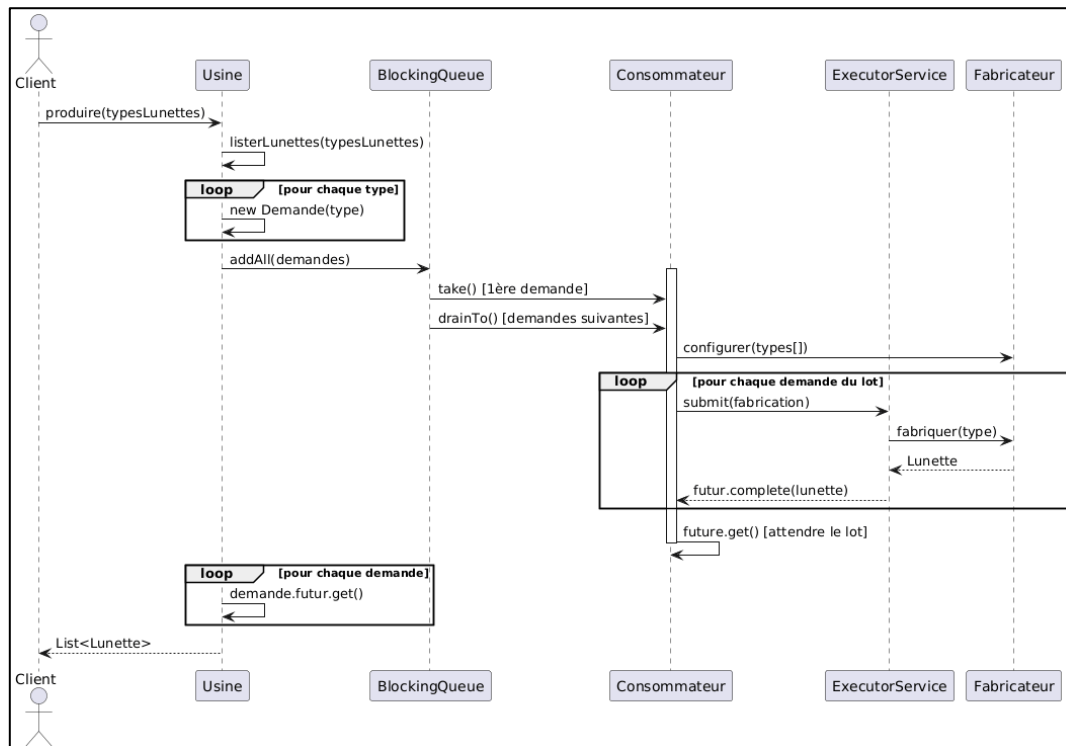


Diagramme de séquence qui résume le fonctionnement de l'usine.

Tests unitaires du back-end

Afin de tester correctement notre back-end, nous avons mis en place des tests unitaires. Ils portent principalement sur la classe MessageParser, qui contient l'essentiel de la logique métier testable sans dépendance externe.

Voici un tableau récapitulatif de tous les tests écrits avec la fonction de chacun :

Méthode de test	Ce qui est testé
parseTypeValide	Un type de lunette connu ne lève pas d'exception
parseTypeInconnuLeveException	Un type inconnu lève IllegalArgumentException avec le bon message
qteLegalZeroAutorise	La quantité 0 est valide pour un type

Méthode de test	Ce qui est testé
qteLegalNeufAutorise	La quantité 9 est valide
qteLegalDixRefuse	La quantité 10 est refusée
qteLegalNegatifRefuse	Une quantité négative est refusée
parsePayloadValide	Un message bien formé retourne la map attendue
parsePayloadFormatInvalideLeveException	Un message mal formé lève une exception
parsePayloadTypeInconnuLeveException	Un type inconnu dans le message lève une exception
parsePayloadQuantiteDixLeveException	Une quantité de 10 lève une exception
parsePayloadQuantiteNegativeLeveException	Une quantité négative lève une exception
parsePayloadTotalZeroLeveException	Un total de 0 lève une exception
serializeLunettesValide	Une lunette est correctement transformée en chaîne
serializeLunettesVide	Une liste vide retourne une chaîne vide

Réponses aux questions du sujet

Comment gérer l'absence d'usine connectée ?

Dans l'état actuel, si aucun back-end n'est connecté au broker, les messages publiés par le client sont perdus. On pourrait mettre place plusieurs approches pour gérer ce cas :

- Permettre au broker de conserver les derniers messages reçus et les transmettre dès qu'un abonné se connecte.
- Implémenter un mécanisme relance coté client pour lui permettre de relancer la commande si aucune confirmation n'est reçue après un certain délai.

Gérer plusieurs commandes en parallèle pour un même client

Actuellement, ce n'est pas prévu que le client puisse effectuer plusieurs commandes en même temps. Pour résoudre ce problème, on pourrait gérer plusieurs listeners MQTT simultanément pour attendre les réponses de plusieurs commandes identifiées par un identifiant unique. L'interface serait à adapter pour afficher l'attente de fabrication et le résultat de plusieurs commandes en même temps.

Gérer plusieurs usines

Si on avait plusieurs usines qui s'abonnent toutes au même topic, chaque commande serait reçue par toutes les usines et ça produirait des doublons. Pour éviter ça, on pourrait implémenter un système de dispatcher et d'identifiant pour les usines pour envoyer les commandes vers celles qui sont les moins saturées. Ou alors donner à chaque usine un et un seul type de lunettes à fabriquer.

Ce qui reste à améliorer

Nous n'avons pas pu implémenter toutes les fonctionnalités coté back-end, voici une liste de celles qui n'ont pas été implémentées :

- Le topic `orders/{id}/status`. Nous n'avons pas eu le temps d'ajouter ce bonus qui permettait informer le client de l'avancement de la fabrication. C'est actuellement uniquement une vue front-end qui affiche un message pendant la fabrication.
- La reconnexion automatique au broker. Actuellement, si la connexion est perdue, le serveur affiche un message mais ne tente pas de se reconnecter.

Partie front-end - Les vues JavaFX

Dans cette partie, nous allons présenter l'interface graphique de l'application, que nous avons développé en JavaFX. Nous avons au préalable réalisé des maquettes afin d'avoir une idée globale de ce à quoi allait ressembler l'application. Cela nous a permis d'accélérer le développement de chaque écran en JavaFX. Notre application permet de passer des commandes de lunettes, suivre leur fabrication et vérifier l'authenticité des numéros de série générés.

Les maquettes

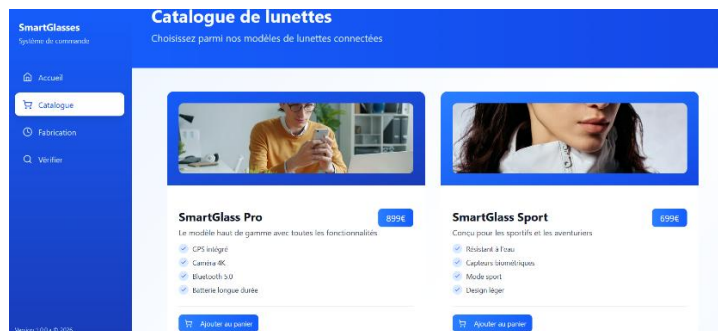
Avant de coder, nous avons réalisé plusieurs maquettes sur Figma pour définir l'apparence de chaque écran de l'application :

- La page d'accueil, avec la présentation de l'application et les boutons pour accéder aux différents pages de l'application.
- La page de catalogue des modèles de lunettes.
- La page de fabrication quand des lunettes sont en cours de production puis quand les résultats sont arrivés.
- La page de vérification des numéros de série générés.

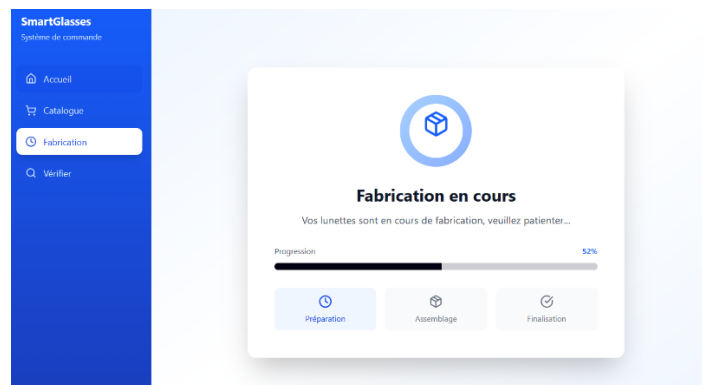
Ces maquettes nous ont servi tout au long du développement pour garder une identité visuelle cohérente et avoir une application un minimum jolie. Voici quelques maquettes que nous avons utilisé :



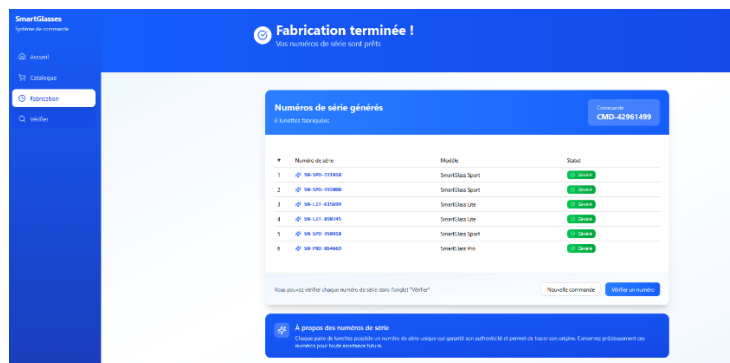
Maquette de l'écran d'accueil de l'application



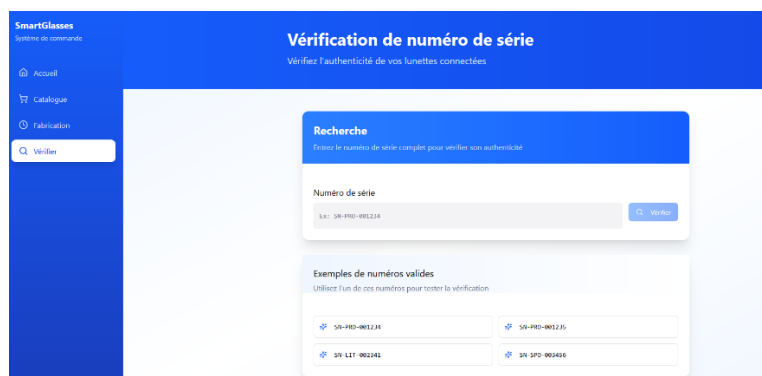
Maquette de l'écran de commande



Maquette de l'écran de fabrication lors de la fabrication



Maquette de l'écran de fabrication quand la commande est reçue



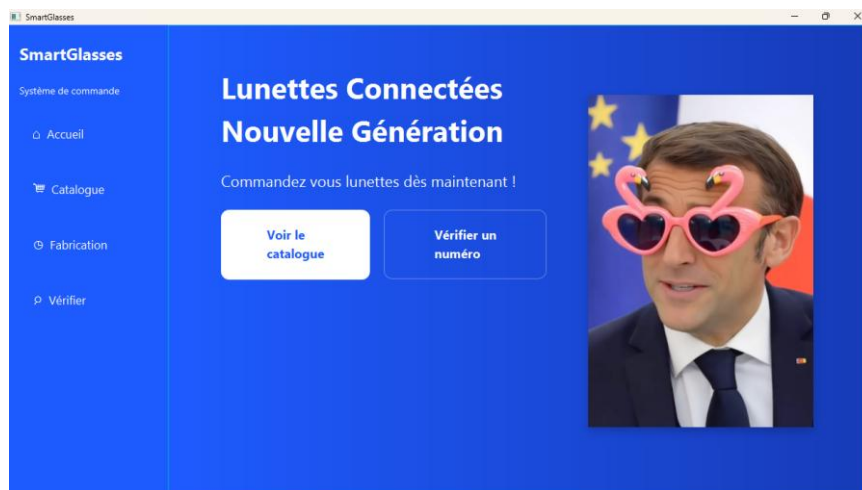
Maquette de l'écran de vérification des numéros de série

Présentation des écrans

La page d'accueil

La page d'accueil présente rapidement l'application et guide l'utilisateur vers les fonctionnalités principales. Elle comprend :

- Une section de titre « Lunettes Connectées Nouvelle Génération ».
- Une image d'illustration chargée depuis les ressources JavaFX.
- Un bouton vers la page du catalogue et un bouton vers la page de la vérification de numéro de série.



Page d'accueil de l'application

La page de catalogue

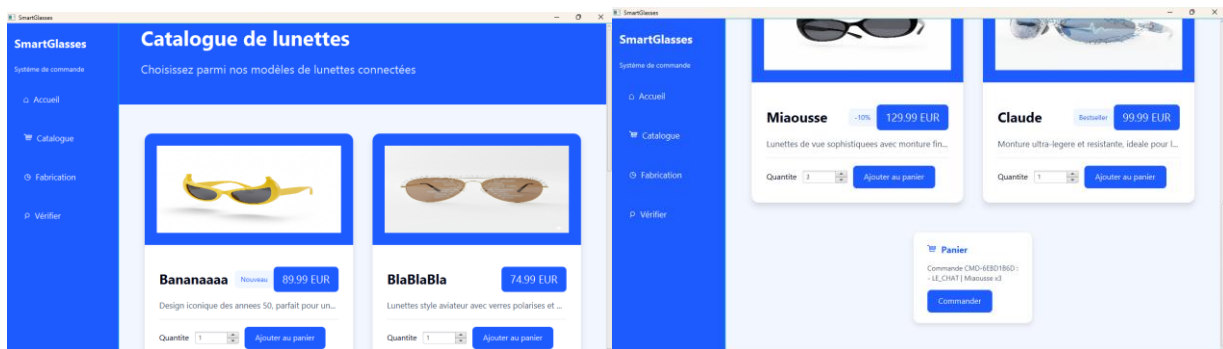
La page du catalogue permet à l'utilisateur de créer une commande contenant la liste des lunettes qu'il souhaite commander avec le nombre de chaque modèle. La page affiche les 4 modèles disponibles :

Identifiant	Nom affiché	Prix	Badge
banana	Bananaaaa	89.99 €	Nouveau
chatgpt	BlaBlaBla	74.99 €	
le_chat	Miaousse	129.99 €	-10%
claudio	Claude	99.99 €	Bestseller

(Les identifiants correspondent aux TypeLunette du Fabricateur)

Chaque carte affiche une image du modèle, son nom, son prix, un badge s'il en possède un, une description, un sélecteur de quantité (de 1 à 9 (inclus) pour correspondre aux attentes du serveur) et un bouton « Ajouter au panier ».

On retrouve en bas de l'écran un panier qui résume les modèles de paires de lunettes et le nombre commandé par l'utilisateur avec un bouton « commander » qui permet de lancer la commande avec les éléments présents dans la liste. A chaque fois que l'utilisateur appuie sur le bouton d'ajout au panier, la liste se met à jour automatiquement.



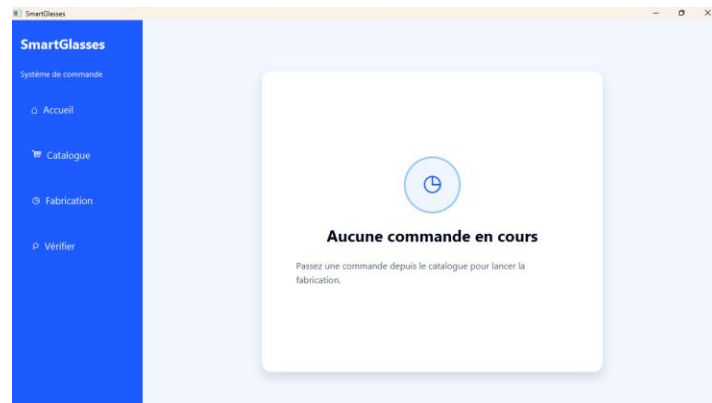
Page de catalogue

La page de fabrication

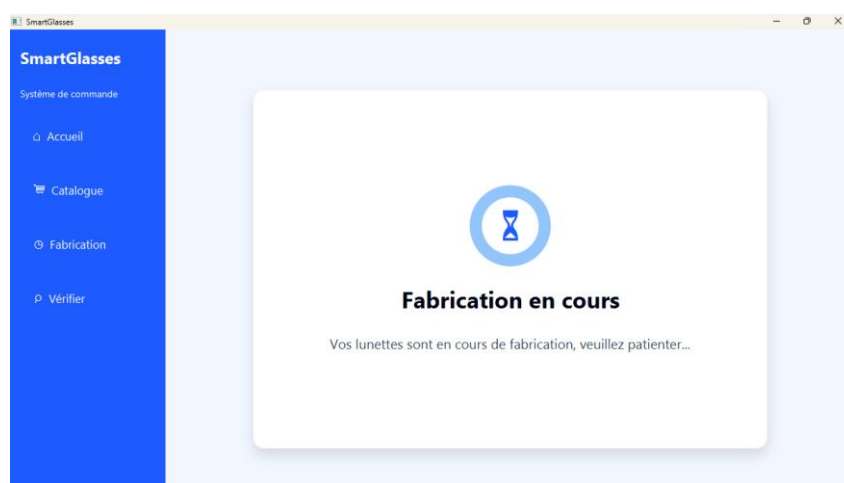
La vue de fabrication comprend trois états différents, quand aucune commande n'est en cours, quand une commande est en cours de fabrication, quand la commande est reçue :

- Aucune commande en cours, la page affiche simplement un message indicatif
- Une commande est en cours de fabrication, un autre message indicatif est affiché, et il est remplacé quand la commande est fabriquée
- Une commande est reçue, on peut retrouver le numéro de la commande ainsi que le détail de tous les modèles fabriqués avec leur numéro de série respectifs. On y retrouve également deux boutons qui permettent de créer une nouvelle commande et accéder à la page de vérification des numéros de série.

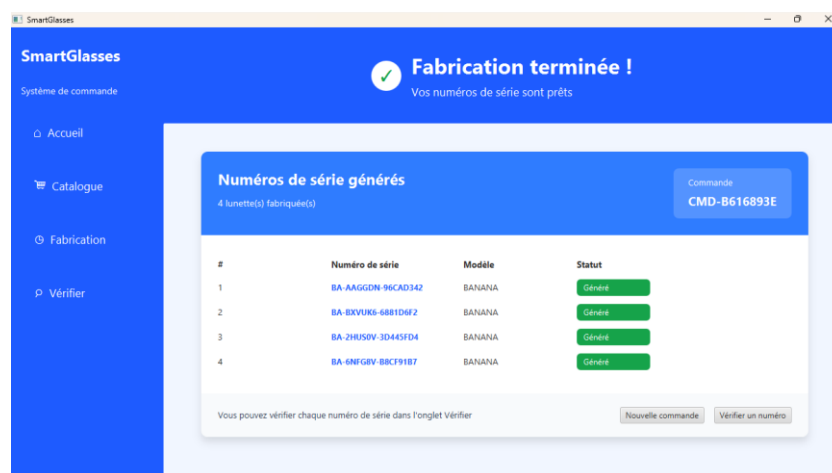
Voici un screen de chaque vue de la page fabrication pour visualiser ce que nous avons réellement implémenté par rapport aux maquettes initialement créées :



Page de fabrication lorsqu'aucune commande n'est en cours



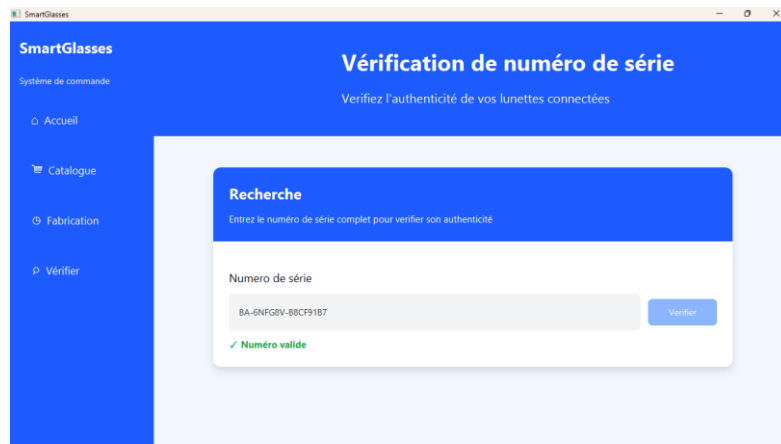
Page de fabrication lorsqu'une commande n'est en cours de fabrication



Page de fabrication lorsqu'une commande est reçue

La page de vérification

La page de vérification permet à l'utilisateur de tester l'authenticité d'un numéro de série. Elle est très simple, elle contient uniquement un champ de saisie, un bouton « Vérifier » et un message indiquant si le numéro est valide ou non.



Page de vérification affichant un numéro de série valide

Articulation des pages et navigation

Les quatre pages de l'application partagent toutes la même fenêtre. L'affichage est géré par la classe Layout qui sert de dispatcher en fonction du bouton cliqué. Nous avons fait ce choix pour simplifier la maintenance car comme ça toutes les redirections sont gérées au même endroit.

La classe Layout

Le Layout est construit une seule fois, au démarrage de l'application. Il définit la structure permanente de la fenêtre : une barre latérale à gauche et une zone de contenu centrale. La barre latérale contient les boutons de navigation (Accueil, Catalogue, Fabrication, Vérifier) et ne change jamais. Seule la zone centrale est remplacée lors de chaque changement de page.

Ce remplacement se fait via la méthode `setContent(Node node)`, qui appelle directement `root.setCenter(node)` sur le `BorderPane` principal. Chaque vue expose une méthode `getView()` qui retourne son nœud JavaFX racine, lequel est passé à `setContent()` pour être affiché.

Le Layout reçoit également une instance d'`AppController` à sa construction et la conserve. Cette référence est transmise aux vues qui en ont besoin, ce qui permet à chaque vue d'accéder aux contrôleurs sans que ceux-ci ne soient instanciés plusieurs fois.

La navigation

Chaque bouton de la barre latérale crée une nouvelle instance de la vue correspondante et l'injecte dans la zone centrale via `setContent()`. Par exemple, le bouton "Catalogue" exécute :

```
« setContent(new CatalogueView(this, appController.getOrderController()).getView()) »
```

Le Layout passe `this` (sa propre référence) à chaque vue lors de son instantiation. Cela permet à la vue de rappeler `setContent()` elle-même lorsqu'elle doit déclencher une navigation interne, sans passer par la barre latérale.

La navigation depuis les vues elles-mêmes

Certaines transitions ne passent pas par la barre latérale mais sont déclenchées par des actions dans les vues elles-mêmes. Ce sont des navigations contextuelles qui s'appuient sur la référence au Layout transmise à la construction. Exemples :

- dans `CatalogueView`, le bouton "Commander" du panier instancie directement une `FabricationView` et l'affiche.
- dans `FabricationView`, une fois la fabrication terminée, la vue se remplace elle-même par son propre état de succès via `showSuccess(layout)`. Les boutons "Nouvelle commande" et "Vérifier un numéro" de cette même page déclenchent respectivement la `CatalogueView` et la `VerificationView`.

Ce mécanisme est simple et cohérent : toute navigation, qu'elle vienne de la barre latérale ou d'une action dans une vue, passe toujours par le même point d'entrée, la méthode `setContent()` du Layout.

Lancement de l'application JavaFX

La classe Lanceur

Cette classe est le point de d'entrée de la JVM, dans cette classe, la méthode `main()` fait juste appel à `Main.main()`. On fait ça pour éviter des soucis quand on met l'application dans un `.jar`. Parce que le manifest ne peut pas pointer sur une classe qui étend `Application`. Donc, sans Lanceur, certains systèmes n'arrivent pas à démarrer notre application.

La classe Main

La classe Main hérite de `javafx.application.Application` et constitue le vrai point d'entrée JavaFX. Sa méthode `main()` appelle `launch()`, qui permet de lancer le démarrage de JavaFX. Et c'est dans `start()` que l'initialisation de l'application a lieu :

1. `AppController` est instancié. Il crée et regroupe les deux contrôleurs utilisés (`OrderController` et `SerialController`).
2. Le `Layout` est instancié avec l'`AppController`. Le `Layout` construit la barre latérale, crée l'`AppController` et charge la page d'accueil par défaut.
3. Une `Scene` est créée à partir du `BorderPane` racine du `Layout`, avec une résolution initiale prédéfinie.
4. La `Scene` est appliquée au `Stage` (la fenêtre de l'application), le titre est défini, et la fenêtre est rendue visible avec `stage.show()`.

Partie front-end - Architecture MVC et communication réseau

Cette partie présente l'architecture interne du module frontend : la séparation MVC, les contrôleurs, les modèles de données et la communication MQTT avec le serveur.

Architecture MVC

Le patron MVC

L'application utilise le patron Modèle-Vue-Contrôleur (MVC). Il permet d'organiser le code en trois couches avec des responsabilités séparées :

- Le Modèle encapsule les données et les règles métier. Il est totalement indépendant de la Vue et ne connaît ni JavaFX ni le réseau.
- Le Contrôleur orchestre la logique applicative. Il reçoit les actions de l'utilisateur via la Vue, manipule le Modèle.
- La Vue affiche les données et capte les interactions de l'utilisateur. Elle délègue toutes les actions au Contrôleur.

Dans le contexte JavaFX, ce patron est particulièrement naturel. Les `StringProperty` et `ObjectProperty` permettent de lier les composants graphiques aux données du Contrôleur de façon réactive, sans que le Contrôleur n'ait besoin de connaître les composants visuels (exemple avec l'actualisation en direct des éléments du panier d'utilisateur).

Organisation des packages

Le package frontend est organisé en plusieurs dossiers, afin de séparer clairement les différentes parties du patron MVC. Voici un rapide bilan du contenu de chaque dossier :

Dossier	Classes
model	Order, GlassesModel, SerialPair
controller	AppController, OrderController, SerialController
service	ClientMqtt
view	Layout, HomeView, CatalogueView, FabricationView, VerificationView

Les classes de Modèle

GlassesModel

Le record GlassesModel contient toutes les informations concernant un modèle de lunette du catalogue. On a décidé d'utiliser un record plutôt qu'une classe car une fois créé, un modèle de lunettes ne change jamais et cela simplifie la définition et l'utilisation de l'objet (pas de getters/setters à écrire par exemple).

Le record contient la définition de la constante « CATALOGUE », une liste contenant tous les modèles de lunettes connus. Cela permet de la rendre accessible partout dans l'application et de la rendre impossible à modifier, ce qui garantit qu'elle contient des données valides.

Order

La classe Order représente une commande en cours de construction ou déjà fabriquée. C'est la classe modèle, la plus complète, la classe modèle centrale de l'application. Elle contient plusieurs attributs :

- Un id, pour identifier de manière unique la commande.
- Une hashMap<GlassesModel, Integer> contenant les modèles de lunette commandés ainsi que leur quantité associée. Ça permet d'assurer qu'un modèle ne peut être présent qu'une seule fois dans la commande.
- Une liste de SerialPair qui contient les modèle et les numéros de série des lunettes fabriquées.

La classe met à dispositions des méthodes publiques pour

- Compléter la commande en cours en ajoutant un nouveau modèle de lunette avec une quantité définie ou modifier la quantité d'un modèle déjà présent : `addGlasses(GlassesModel model, int quantity)`
- Afficher la commande et son contenu de manière organisée : `toString()`
- Des getters/setters pour accéder et modifier les différents attributs de la classe.

Quelques précisions supplémentaires sur le fonctionnement de certaines méthodes de la classe. La méthode `addGlasses(model, quantity)` utilise `Map.put()`, cela permet d'ajouter le modèle s'il n'est pas présent dans la liste ou de mettre à jour sa quantité s'il est déjà présent. Et setter

setSerialNumbers protège contre les valeurs null en ajoutant une liste vide si le paramètre donné est null. Pour toujours avoir une instance de liste à retourner et éviter les erreurs du type NullPointerException.

SerialPair

La classe SerialPair représente une paire (modèle, numéro de série) générée lors de la fabrication. On utilise là aussi un record, car un numéro de série d'un modèle ne peut plus être changés une fois fabriqués.

Les Contrôleurs

AppController

AppController est le point d'entrée unique pour les contrôleurs. On a décidé d'implémenter le patron de conception Façade, il permet de regrouper les deux contrôleurs spécialisés et d'exposer une interface simple à la Vue. La Vue reçoit une instance d'AppController à sa construction et la partage aux vues enfants. Cela permet de réduire le couplage, Vue ne dépend que d'AppController et non directement des contrôleurs spécialisés. On peut donc plus facilement en ajouter des nouveaux.

OrderController

Ce contrôleur permet de gérer tout le cycle de vie d'une commande, de sa création, en passant par sa fabrication, jusqu'à la réception des numéros de série des modèles produits. Pour cela, la classe dispose de plusieurs attributs et méthodes que nous allons présenter ci-dessous.

Concernant les attributs, la classe n'en possède que 2, un attribut de type Order pour conserver la commande courante, et un attribut de type StringProperty pour afficher de manière dynamique le contenu de la commande en cours.

Concernant les méthodes de cette classe, il faut partir du principe qu'une commande n'a de sens que si elle contient au moins un modèle de lunette et une quantité. C'est pourquoi on a une méthode « addGlasses » qui vérifie si une commande est déjà en cours pour y ajouter le modèle et la quantité ou en créer une nouvelle si aucune n'est créée.

Dans ce second cas, elle appelle « createOrder » qui attend un modèle et une quantité en paramètre. Elle permet de créer une nouvelle commande avec un identifiant basé sur un UUID

aléatoire. Les 8 premiers caractères sont extraits pour obtenir un identifiant lisible du type CMD-1A2B3C4D. Les paramètres renseignés y sont ajoutés puis la commande est retournée, prête à être utilisée.

Une fois les informations ajoutées à la commande courante, on actualise les données de la commande qui sont affichées à l'aide de la méthode « refreshInfos() » liée à l'attribut StringProperty.

Mais la méthode principale de la classe reste « startFabrication() » qui permet de lancer la fabrication de la commande courante. Elle instancie un ClientMqtt, construit le message à envoyer au serveur à partir de la commande courante, puis l'envoie via MQTT et attend la réponse. Une fois la réponse reçue, elle analyse les numéros de série retournés par le serveur, les stocke dans la commande courante sous forme de liste de SerialPair, puis appelle « refreshInfos() » pour mettre à jour l'affichage. La connexion MQTT est systématiquement fermée en fin d'exécution, qu'elle se termine normalement ou avec une erreur. Cette méthode étant bloquante en attendant la réponse du serveur, elle est toujours appelée depuis un thread dédié afin de ne pas figer l'interface graphique et pouvoir afficher la page qui fait patienter l'utilisateur.

Voici un diagramme de séquence de la méthode « startFabrication() » qui permet de mieux visualiser le fonctionnement de la méthode :

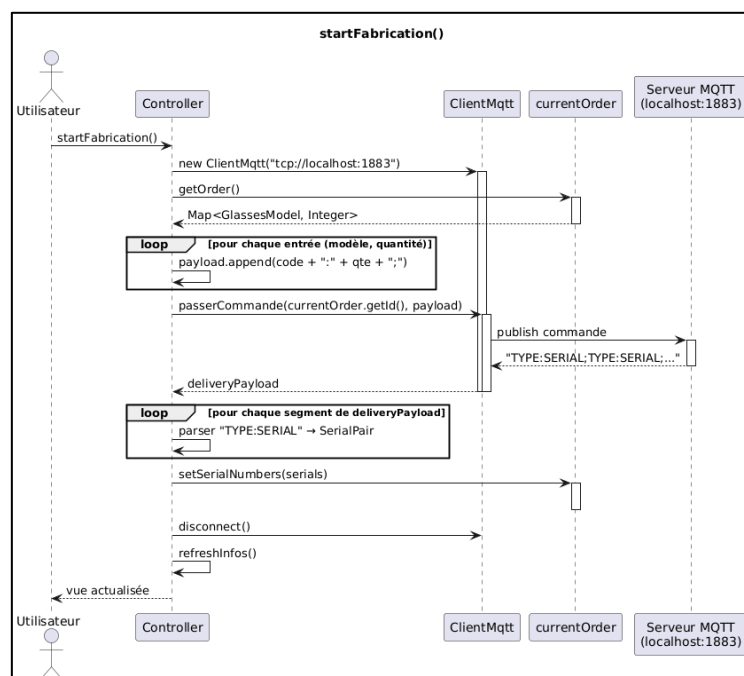


Diagramme de séquence de la méthode « startFabrication() »

SerialController

Ce contrôleur est plus léger, il ne contient qu'une méthode et permet uniquement de vérifier auprès du serveur si un numéro de série renseigné est correct. Pour cela, il se connecte au client MQTT, envoie sa demande de vérification et attend la réponse de manière asynchrone. Il va ensuite la renvoyer à la vue qui va pouvoir afficher le résultat à l'utilisateur. Sans oublier de déconnecter le client une fois la réponse reçue.

La service ClientMqtt

Présentation du protocole MQTT

MQTT est un protocole de messagerie publication/souscription conçu pour les environnements à faible bande passante. Un broker centralise tous les messages. Les clients publient sur des topics et s'y abonnent pour recevoir des messages. Ce modèle est bien adapté pour notre projet car il sépare temporellement le front-end du back-end.

Synchronisation par BlockingQueue

Le principal défi d'implémentation du client MQTT que nous avons rencontré est la synchronisation entre le thread appelant (qui attend une valeur de retour) et le callback « `messageArrived()` ».

La solution que nous avons choisie utilise une « `BlockingQueue<String>` », qui suit le patron Producteur/Consommateur :

1. Le thread appelant publie le message MQTT, puis appelle « `queue.poll(timeout)` » pour se mettre en attente.
2. Quand le broker reçoit la réponse du serveur, il appelle « `messageArrived()` » sur le thread Paho.
3. « `messageArrived()` » dépose la réponse dans la queue avec « `queue.offer(payload)` ».
4. Le thread appelant, débloqué par l'arrivée du message, récupère et retourne la valeur.

Nous avons utilisé deux queues distinctes, pour les commandes et pour les vérifications, afin d'éviter des confusions en cas d'utilisation simultanée. Avant chaque appel, la queue est nettoyée pour éliminer d'éventuelles réponses résiduelles.

Gestion des timeouts

Nous avons, à chaque utilisation, défini la méthode « poll() » avec un timeout pour ne pas créer de deadlock :

- 180 secondes pour la fabrication, nous avons vu après quelques tests que la production prenait un certain temps.
- 10 secondes pour la vérification, les vérifications sont généralement très rapides.

Si le timeout expire (poll retourne null), une exception est levée avec un message descriptif, affiché à l'utilisateur.

Tests unitaires du front-end

Stratégie de test

Classe	Type de test	Framework	Nombre de tests
Order	Unitaire pur	JUnit 5	17
OrderController	Unitaire + mock	JUnit 5 + Mockito	21
SerialController	Intégration partielle	JUnit 5	2
Vues JavaFX	Tests fonctionnels	—	—

Tests de Order

La classe Order n'a aucune dépendance externe. Les tests sont plutôt simples et couvrent tous les comportements documentés : l'identifiant de commande, le comportement upsert de « addGlasses », la gestion de plusieurs modèles, la protection contre les valeurs null, et le formatage de « toString ».

Tests de OrderController

La classe OrderController instancie un ClientMqtt directement dans « startFabrication() ». Pour tester cette méthode sans serveur MQTT disponible, on a décidé d'utiliser un « mockConstruction » de Mockito, qui intercepte l'appel au constructeur et retourne un mock à la place. Cela nous a permis de tester plus précisément cette classe sans devoir lancer un serveur de

test. Pour cela nous avons dû ajouter une nouvelle dépendance Maven dans le pom.xml du front-end pour accéder à la librairie Mockito.

Concernant les tests que nous avons effectués, nous avons vérifié que : la vérification des numéros de série est bien parsé, que la connexion est fermée après la commande, que le binding réactif est mis à jour, et que les entrées malformées sont bien ignorées.

Tests de SerialController

Concernant les tests de la classe SerialController, nous en avons écrit qui couvrent les cas suivants : la vérification que la méthode retourne bien un CompletableFuture, et la vérification que l'absence de serveur produit une RuntimeException avec le bon message.

Qualité du code

Javadoc

Toutes les classes publiques et méthodes importantes disposent d'une documentation Javadoc avec les balises @param, @return et @throws si nécessaire.

Principes de conception appliqués

Afin d'écrire du code le plus proprement possible, nous avons appliqué quelques bonnes pratiques de développement :

- Séparation des responsabilités (SRP) : chaque classe a une responsabilité unique et clairement définie.
- Couplage faible : la couche Contrôleur ne connaît pas la couche Vue. La communication se fait uniquement via des propriétés observables et des CompletableFuture.
- Immuabilité : les records GlassesModel et SerialPair sont immuables. Le catalogue est déclaré comme List.of() immuable.

Difficultés rencontrées

Synchronisation MQTT et thread appelant

La principale difficulté technique que nous avons rencontrée vient du fait que MQTT fonctionne de manière asynchrone. Cela signifie que lorsque l'on envoie un message au serveur, on ne reçoit pas la réponse immédiatement à la suite dans le code : c'est une fonction appelée « callback », nommée « `messageArrived()` », qui est automatiquement déclenchée par la bibliothèque Eclipse Paho lorsque la réponse arrive, et ce dans un thread complètement séparé de celui qui a envoyé le message. Le problème est que notre contrôleur, lui, a besoin d'attendre cette réponse avant de pouvoir continuer son exécution. Il fallait donc trouver un moyen de faire communiquer ces deux threads entre eux. La solution que nous avons choisie est l'utilisation d'une `BlockingQueue` : le thread principal envoie le message puis se met en attente sur la queue, et lorsque « `messageArrived()` » est appelé, il dépose la réponse dans cette même queue, ce qui débloque automatiquement le thread en attente. Un timeout est également défini pour éviter d'attendre indéfiniment si le serveur ne répond pas.

Binding réactif sans couplage

Une autre difficulté que nous avons rencontrée est le fait de maintenir l'affichage à jour dans la Vue sans que le Contrôleur n'ait besoin de connaître les composants graphiques. Pour résoudre ce problème, nous avons utilisé une `StringProperty`, qui est un type JavaFX particulier permettant à d'autres composants de « s'abonner » à sa valeur et d'être automatiquement notifiés dès qu'elle change. Il suffit que le Contrôleur mette à jour cette propriété lorsque la commande évolue, et le label dans la Vue, lié à cette propriété via « `textProperty().bind()` » se met à jour également.

Mock de construction dans les tests

Une dernière difficulté que nous avons rencontrée concerne les tests de la méthode « `startFabrication()` » de la classe `OrderController`. Le problème est que cette méthode crée elle-même une instance de `ClientMqtt` en interne, ce qui signifie qu'au moment du test, elle aurait tenté de se connecter à un vrai serveur MQTT qui n'est pas disponible dans un environnement de test. Modifier le code de production uniquement pour faciliter les tests n'étant pas une bonne pratique, nous avons donc utilisé un package appelé Mockito et notamment sa méthode appelée « `mockConstruction()` ». Celle-ci permet d'intercepter l'appel au constructeur de `ClientMqtt` et de retourner à la place un faux objet que l'on contrôle entièrement, sans avoir besoin de modifier la méthode testée.

Intégration continue

Cette partie présente la mise en place de l'intégration continue (CI/CD) du projet, réalisée avec GitHub Actions.

Objectif

L'objectif de l'intégration continue est d'automatiser la compilation et la publication du module factory, c'est-à-dire éviter d'avoir à effectuer ces étapes manuellement à chaque modification. Concrètement, à chaque fois qu'une nouvelle version du projet est publiée sur GitHub, une action automatique est déclenchée. Cette action se charge de compiler le code du module factory et de produire un fichier .jar, qui est l'équivalent d'un exécutable en Java, contenant l'ensemble du module prêt à être utilisé. Ce fichier est ensuite déposé automatiquement sur GitHub Packages, qui est un espace de stockage proposé par GitHub permettant de mettre à disposition des bibliothèques et modules compilés, afin que les autres modules du projet puissent le récupérer facilement sans avoir à le compiler eux-mêmes.

Fonctionnement de la GitHub Action

La GitHub Action est déclenchée automatiquement lorsqu'une release est publiée sur le dépôt GitHub du projet. Elle exécute les étapes suivantes :

1. Récupération du code source depuis le dépôt (git checkout).
2. Installation de Java (JDK 17) dans l'environnement d'exécution.
3. Compilation du module factory avec Maven (mvn package).
4. Publication du .jar sur GitHub Packages pour qu'il soit accessible comme dépendance Maven.

Le fichier de configuration de la GitHub Action est un fichier YAML placé dans le dossier .github/workflows/ du dépôt.

Intégration avec Maven

Le module server déclare le module factory comme dépendance dans son fichier pom.xml. Grâce à GitHub Packages, Maven peut télécharger automatiquement la dernière version compilée du module factory lors de la compilation du module server. Cela évite d'avoir à partager manuellement le fichier .jar entre les membres de l'équipe.

Action de compilation continue

Nous avons également mis en place une seconde action GitHub qui répond à un besoin différent. Plutôt que de publier un module sur GitHub Packages, l'objectif ici est de vérifier automatiquement que l'ensemble du projet compile correctement et de rendre les fichiers .jar produits facilement téléchargeables. Cette action se déclenche à chaque fois qu'un nouveau code est poussé sur la branche principale du dépôt, ce qui permet de détecter immédiatement si une modification a cassé la compilation.

Concrètement, l'action effectue les étapes suivantes : elle commence par récupérer le code source du dépôt, puis installe Java 21 dans l'environnement d'exécution. Elle compile ensuite les deux modules du projet séparément, d'abord le module server qui correspond au back-end, puis le module frontend qui correspond à l'interface graphique. Pour chaque compilation, Maven se charge automatiquement de récupérer les dépendances nécessaires, dont le module factory publié sur GitHub Packages par la première action. Une fois les deux modules compilés avec succès, les fichiers .jar produits sont déposés en tant qu'artefacts sur GitHub, ce qui signifie qu'ils sont directement téléchargeables depuis la page de l'action sur le site de GitHub, sans avoir besoin de cloner et compiler le projet soi-même.

Bénéfices pour le projet

La mise en place de l'intégration continue permet de simplifier le travail en équipe. Antoine, qui développe le module server, peut récupérer automatiquement la dernière version de l'Usine sans avoir à la compiler lui-même.

Bilan du projet

Ce qui a été réalisé

Ce projet nous a permis de développer une application distribuée complète, en mobilisant de nombreux concepts vus en cours et en TDs :

- Architecture MVC avec JavaFX : séparation claire des responsabilités, data-binding via les propriétés observables.
- Usage des records pour les classes de valeur immuables, List.of() pour les collections immuables.
- Communication réseau asynchrone : intégration du protocole MQTT côté serveur et côté client.
- Programmation concurrente : CompletableFuture, ExecutorService, Platform.runLater(), gestion des threads JavaFX.
- Tests unitaires de qualité : pour le back-end, pour le front-end, et mock de construction avec Mockito.
- Intégration continue : publication automatique du module factory sur GitHub Packages.

Les difficultés rencontrées

Nous avons rencontré de nombreuses difficultés lors du développement de ce projet. Voici celles que nous avons eu le plus de mal à résoudre :

- La synchronisation entre les threads MQTT et les threads JavaFX, résolue grâce à la BlockingQueue et à « Platform.runLater() ».
- La mise en place du mock de construction de Mockito pour tester les méthodes quiinstancient directement un client MQTT.
- La gestion de la concurrence dans l'Usine pour permettre à plusieurs commandes d'être traitées simultanément.

Ce qui pourrait être amélioré

Certains points de l'application pourraient encore être améliorés, notamment :

- Implémenter le topic `orders/{id}/status` pour informer le client de l'avancement de la fabrication en temps réel.
- Ajouter la reconnexion automatique au broker en cas de perte de connexion.
- Écrire des tests d'intégration simulant un échange MQTT complet de bout en bout.
- Ajouter une persistance des commandes (par exemple en base de données) pour survivre à un redémarrage du serveur.

Répartition du travail

La répartition du travail a bien fonctionné dans l'ensemble. Chaque membre de l'équipe a travaillé sur une partie indépendante, ce qui a limité les conflits. La séparation entre les vues et l'architecture MVC/contrôleurs et le serveur nous a permis de travailler chacun de notre côté avant de tout mettre en commun.

Conclusion

Ce projet nous a permis de mettre en pratique des concepts avancés de Java dans un contexte réaliste et proche d'une application professionnelle. Des sujets comme la communication par MQTT, la gestion de la concurrence, le data-binding JavaFX ou encore les tests avec Mockito sont des notions que nous avons abordées en cours mais que nous n'avions pas encore eu l'occasion d'utiliser dans un projet concret. Les mettre en œuvre ensemble, dans une application distribuée complète avec un front-end, un back-end et un module de fabrication, nous a permis de mieux en comprendre les enjeux réels et les difficultés que l'on ne perçoit pas forcément sur un exercice isolé.

Ce projet nous a également montré l'importance de bien séparer les responsabilités dans le code. Grâce à l'architecture MVC et à la séparation en modules indépendants, chaque membre de l'équipe a pu travailler sur sa partie sans bloquer les autres, et les tests ont été plus simples à écrire car chaque classe avait un rôle clair et délimité.